

Agentic Engineering: Five Papers

A pragmatic reader's guide to five arXiv papers from 28 May 2026 — search honesty, self-specializing agents, multi-agent world models, agent aging, and memory observability

[papers-to-content](#) · [arXiv digest 2026-05-28](#)

May 2026

Chapter 0: The Through-Line — What This Week's Papers Agree On

Five papers, five labs, five different problems: whether search agents really search, how to specialize a small computer-use agent, how to model many agents in a shared world, how deployed agents decay, and how to trace a memory error to its source. Read them back to back and a theme assembles itself without anyone planning it. This is a week obsessed with what happens *after* the demo — once an agent is deployed, aging, remembering, and quietly failing in ways a launch-day benchmark will never show you. This chapter names the shared convictions; the five deep dives follow.

Static, day-one evaluation is hiding your real failures

The most pointed agreement runs between LiveBrowseComp and AgingBench, and it's an indictment of how we measure agents. **LiveBrowseComp** shows that a frozen browsing benchmark quietly rewards a model for *knowing* the answer rather than *finding* it — strip the tools and frontier models still answer up to 44.5% from memory, so the leaderboard is partly a memory test wearing a search costume. **AgingBench** shows the opposite blind spot on the time axis: an agent that aces every test on launch day can have its factual precision collapse from 0.90 to 0.37 over six weeks while its behavioral dashboards stay spotless. One paper defeats the shortcut with recency; the other defeats it with longevity. Both are saying the same thing — a single snapshot score is a flattering lie, and the failures that actually reach your users live in the conditions a static eval erases.

The borrowable instinct: evaluate under conditions that *defeat the shortcut*. Test on facts younger than your training cutoff; test reliability as a curve over sessions, not a pass/fail on day one.

The failure is in an operation, not “the model”

When something goes wrong, three of these papers refuse the lazy verdict that the model hallucinated and instead localize the fault to a specific operation. **MemTrace** turns a memory pipeline into a graph and walks it upstream to the *earliest decisive faulty operation* — the write three sessions ago, not the answer you're staring at. **AgingBench** runs a counterfactual ablation ladder (oracle retriever, oracle context) to partition an error into write-time omission, retrieval interference, or utilization failure — the same wrong answer routed to a different repair. **LearnWeak** splits an agent's mistakes into planning errors (chose the wrong action) versus execution errors (right action, botched parameters) and masks supervision to the span actually at fault.

The unifying move is diagnostic altitude: don't fix the agent, fix the operation. “The agent forgot” is not a root cause — *which* operation, in *which* stage, introduced the fault? Every one of these papers builds machinery to answer that, because a generic fix aimed at a mislocated cause is wasted effort.

Targeting beats scale — again

The brute-force instinct loses in paper after paper. **LearnWeak** tried flooding a small agent with bulk synthetic data and got almost nothing; mining the narrow slice of tasks the agent actually fails is what moved it 11 points. **AgingBench** found that “give it more memory” is the wrong prescription in most aging scenarios, and that bigger models don't fix derived-state tracking — the failure is representational, not a capacity shortfall. **Gamma-World** refuses all-to-all cross-agent attention and routes interaction through a small shared hub, trading quadratic cost for linear with no loss in fidelity. More data, more memory, more attention, more parameters — each paper found the lever was somewhere more surgical.

Memory is where deployed agents actually break

Step back and notice what most of these papers are *about*. LiveBrowseComp is about parametric memory masquerading as search. AgingBench is about the memory harness — compression, interference, revision — decaying over a lifespan. MemTrace is about attributing failures inside a memory pipeline. Even LearnWeak is about an agent's persistent skill deficits, and Gamma-World's whole architecture exists to maintain a coherent shared state across agents and across time. The model weights are increasingly *not* the interesting variable. The interesting variable is the state wrapped around the model: what it stored, what it retrieved, what it compressed away, what went stale, what it can recover when the answer isn't already in its head.

That's the throughline worth carrying into the chapters. Last week's batch was about making agents *do more* — reach for tools, coordinate, deploy to the edge. This week is the sober follow-up: the same agents, observed over time and under memory pressure, and the engineering discipline — honest evals, operation-level attribution, targeted repair — required to keep them reliable once the demo is over.

What to carry into the five chapters

Read the rest with three questions. *Would my eval catch this failure, or does it only test day one against a shortcut the agent can exploit? When my agent fails, can I localize the fault to an operation — or do I just blame “the model” and reach for a bigger one? Is the state wrapped around my model — its memory, its retrieval, its compression — observable, or is it a black box I can only apologize for?*

We open with the most uncomfortable of them: the search agent that isn't really searching at all.

Chapter 1: LiveBrowseComp — Are Search Agents Actually Searching?

Paper: [LiveBrowseComp: Are Search Agents Searching, or Just Verifying What They Already Know?](#)
· arXiv 2605.28721 · **Dataset:** huggingface.co/datasets/Forival/LiveBrowseComp

Watch a top search agent crush a hard browsing benchmark and you'd assume it's doing what the name promises: fanning out across the web, chasing leads, stitching evidence into an answer. This paper makes a quietly devastating case that a lot of that performance is theater. Strip the tools away entirely and frontier models still answer up to **44.5%** of BrowseComp questions from memory alone. The web isn't where the answer comes from. It's where the model goes to nod along with a guess it already made.

The authors give the pattern a name — **Intrinsic Knowledge Dependence**, or IKD — and the rest of the paper is a careful demolition: first proving the shortcut exists, then building a benchmark that closes it.

The shortcut, in three experiments

The diagnostic setup is clean enough that you could replicate it on your own agent this afternoon. Three questions, each one peeling back a layer.

First, *how much can the agent answer with all tools removed?* Closed-book pass@4 across four benchmarks (BrowseComp, BrowseComp-ZH, HLE, GAIA) averaged 38.9, ranging as high as 62. That's the floor of how much "search" performance is really just recall.

Second — and this is the result that should make you uncomfortable — *what happens when the search interface stays live but every answer-supporting document is removed?* You'd expect a robust agent to shrug, find nothing useful, and fall back on its best internal guess. Instead, every model got *worse than its closed-book baseline*. Average pass@4 fell from 26.1 to 6.2. MiniMax M2.5 cratered from 44.5 to 8.0. Searching with no real evidence available is actively worse than not searching at all. The retrieval loop doesn't just fail to help — it drags the agent off a correct hunch and into a thicket of hard negatives.

Third, *where do the agent's queries actually come from?* Tracing each query to where its key terms first appear, the authors found that **more than half of all queries are model-originated** — seeded by the model's own reasoning, not by anything it retrieved — and that share climbs past 60% as browsing drags on. Worse, when answer-supporting evidence *did* surface in the results, agents used it less than a third of the time. The loop is model-led, not evidence-led. The agent decides what's true, then shops for confirmation.

Put those together and you get IKD’s signature: the agent guesses from parametric memory, treats search as a confirmation channel, and collapses the moment the needed fact sits outside what it already knew. Crucially, this is *not* the same as data contamination. Even a question that never appeared in training can be answered through broad world knowledge. As models get more knowledgeable, fixed benchmarks quietly reward breadth of memory and call it search skill.

Building a benchmark memory can’t cheat

If the disease is “the answer is already in the weights,” the cure is questions whose answers *can’t* be. That’s **LiveBrowseComp**: 335 human-authored questions whose answers hinge on facts published within the 90 days before the benchmark was built, pulled from six continuously-updated structured sources — global news (GDELT), film and TV (TMDB), video games (RAWG), cybersecurity disclosures (CVE/NVD), sports results (SportsDB), and earthquakes (USGS).

Two design choices do the heavy lifting. *Recency* pushes facts past the training-data horizon. *Obscurity* filters out globally salient events, because a big enough story gets absorbed into model weights within days through post-training updates. A third filter throws out anything whose answer drifts over time — cumulative box office, live standings, chart positions — so each question has one stable answer for the benchmark’s lifespan. Questions then run a gauntlet: annotators build a full evidence chain, a separate team re-verifies correctness and uniqueness against an LLM-generated pool of decoy answers, three independent solvers must *fail* to crack it within 30 minutes for it to count as hard enough, and verifiers confirm no pre-window substitute evidence exists.

The payoff is stark. On LiveBrowseComp, every model scored **below 2% closed-book** — the memory shortcut is essentially gone. Search-augmented scores dropped 25–40 points relative to BrowseComp. And the leaderboard scrambled: GLM 5.1 led BrowseComp at 68.0 but fell to 33.9, while DeepSeek v3.2, near the bottom of BrowseComp, climbed past several models that had outranked it. Static-to-live rank correlation slipped from a Spearman ρ of 0.87 (BrowseComp vs. its Chinese sibling) to 0.74, and Pearson from 0.79 to 0.53 — the models that benefited most from memory had the least to show once it was taken away.

The honest-broker move is the human baseline: people solved 30% of BrowseComp and 31% of LiveBrowseComp, with near-identical time distributions. The benchmark isn’t harder for a genuine searcher. It’s only harder for an agent that was secretly leaning on recall. One more tell: search-turn distributions shift from a tight cluster of one-or-two-turn “confirm it” trajectories on BrowseComp to a single peak at much higher turn counts on LiveBrowseComp. Without a memory anchor, each query finally has to *advance* the investigation.

What to try on your own agent

The diagnostics are the gift here, more than the benchmark. A few you can run this week:

- **Measure your own closed-book floor.** Run your agent on its eval suite with tools disabled. Whatever it scores is performance you can't attribute to retrieval — and a hidden ceiling on how much your search harness is actually contributing.
- **Block the evidence, keep the interface.** Swap your retrieval index for one stripped of answer-supporting documents but full of plausible distractors. If accuracy drops *below* the closed-book number, your agent is being pulled off correct intuitions by noisy results — a sign it can't recognize when search has failed.
- **Trace query provenance.** For each query, ask whether its key terms first appeared in retrieved text or in the model's own reasoning. A high model-originated rate, climbing over a session, means you've built a confirmation engine, not a discovery engine.
- **Test on facts younger than your training cutoff.** Recency plus obscurity is a cheap, renewable filter against the IKD shortcut — and a reason to favor dynamic, time-windowed evals over a frozen question bank that silently decays as models get smarter.

The deeper lesson for anyone shipping a research agent: a high benchmark score may be measuring how much your model *knows*, not how well it *finds*. Those are different products, and only one of them helps a user with a question the model has never seen.

If LiveBrowseComp asks whether agents can search when memory runs out, the next chapter asks how a small agent gets better at all — not by retraining on more data, but by mining its own failures to specialize itself.

Chapter 2: Learn from Weaknesses — Specializing Small Computer-Use Agents

Paper: [Learn from Weaknesses: Automated Domain Specialization for Small Computer-Use Agents](#) · arXiv 2605.28775 · **Project page:** learnweak.github.io

The obvious way to make a small agent better at a piece of software is to flood it with more training data for that software. The KAIST, Samsung, and DeepAuto.ai team behind LearnWeak tried exactly that first, and found it barely moved the needle. Pour a large synthetic corpus into a 7- or 8-billion-parameter computer-use agent and you get a couple of points, if that. The data the agent needs most is not more of everything. It is the narrow slice of tasks the agent is currently failing.

That reframing is the whole paper. Instead of asking “what does this domain contain,” LearnWeak asks “where does this specific student break,” and builds training data around the answer. No human annotation required.

Why small agents need this in the first place

A computer-use agent perceives a screen, reasons, and clicks or types its way through a task. The strong ones are large proprietary models like Claude Sonnet 4.6 or GPT-5.4. The practical ones are small open models like **EvoCUA** and **OpenCUA**, cheap enough to run on edge devices or inside privacy-sensitive enterprises that can’t ship screenshots to an API. The catch is a stubborn capability gap, worst in domain-specific software with its own conventions and unfamiliar workflows.

You can’t just train a small model on every domain at once. They lack the capacity, and mixing heterogeneous software tends to cause catastrophic forgetting, where learning GIMP quietly degrades LibreOffice. So you specialize, one domain at a time. The question is how to do that without paying for human-labeled trajectories, which are brutally expensive given how long a computer-use task runs.

Mining your own failure modes

LearnWeak runs in two stages, and the first is where the central idea lives. It needs only a handful of seed tasks per domain — the authors used ten, built over six environment configurations, less than two hours of human setup — plus a stronger **reference agent** acting as teacher and an automatic verifier that checks whether a task succeeded.

The loop is a teacher-student bake-off. Run both agents on the same task in the same environment. The verifier scores each. Then collect only the tasks where the teacher succeeds but the student fails. That filter matters more than it looks: because the teacher succeeded, the task is feasible and

the environment is valid, so the failure is genuinely the student’s deficiency and not a broken setup. Those failures get summarized into a weakness report — recurring patterns like picking the wrong operation, mislocating an element, or passing bad arguments to an action.

That report then steers the next round of task generation. A query generator, grounded in representative screenshots pulled from the actual trajectories, synthesizes new tasks aimed at the diagnosed weaknesses, balanced with some open-ended exploration so the dataset doesn’t collapse onto a single failure. Each iteration nudges the task distribution toward whatever the student still can’t do, while exploration keeps it honest. The output is a compact, targeted dataset, not a bulk dump.

The ablations make the case sharply. Iterative generation *without* the weakness report doesn’t beat a one-shot baseline — exploration alone wanders. And the data is student-specific: train EvoCUA-8B on weakness reports derived from a *different* student and the gains shrink. The most useful supervision for an agent comes from that agent’s own mistakes, not a generic notion of “hard tasks.”

Not all errors deserve the same correction

The second stage trains on those failures, and introduces the paper’s other useful idea: an **error-aware** objective that refuses to treat every mistake identically.

The mechanism is step-level preference learning built on DPO. Rather than retraining whole trajectories, LearnWeak replays the teacher’s successful path step by step, queries the student at each step, and keeps only the steps where the two diverge. Each kept step becomes a preference pair: teacher action preferred, student action rejected. Then it splits the divergence into two types. If the student chose the wrong *kind* of action — a different function entirely — that’s a planning error. If it chose the right action but botched the *parameters*, like clicking slightly off-target, that’s an execution error. A token-level mask routes supervision to the part of the response actually responsible, so the update sharpens the weak span instead of uniformly relearning a sequence the student already mostly had right.

The numbers earn the design. Across eight OSWorld domains, LearnWeak lifts EvoCUA-8B by 11.6 points on average and OpenCUA-7B by 11.1, and the specialized 8B student beats its own 32B teacher on GIMP, Thunderbird, and VSCode — corrective supervision turning out to be more than imitation. Standard SFT and plain DPO add little; the error-aware version beats vanilla DPO by 9.62 points. Masking only planning errors or only execution errors falls short. You need both preference learning and the targeted split.

What to take from it

If you specialize or distill small models — agents or otherwise — the operative lesson is cheaper than it sounds. Before you scale data, instrument failure. Stand up a stronger reference, run it head-to-head with your model on the same tasks, and harvest the cases where the strong one wins and

yours doesn't. That gap *is* your curriculum, and it's annotation-free as long as you have a verifier that can call success or failure. Then resist uniform supervision: separate "chose the wrong move" from "executed the right move badly," because they want different corrections, and a loss that blurs them wastes signal on behavior the model already had.

Two honest caveats from the authors. It leans on a teacher reliable enough in the target domain to generate clean successful trajectories — a biased teacher poisons the well. And it assumes a student that already has baseline competence; this repairs domain-specific weaknesses, not missing foundational skills. The modular LoRA design points at the deployment payoff: a library of per-domain adapters over one shared base, swapping in the right specialist per application.

Where LearnWeak makes an agent stronger by studying its failures against a stronger reference, the next chapter asks a different question — not how to train one agent better, but how to model many agents at once. NVIDIA's Gamma-World pushes a generative multi-agent world model past the usual two-player ceiling.

Chapter 3: Gamma-World — World Models

Past Two Players

Paper: [Gamma-World: Generative Multi-Agent World Modeling Beyond Two Players](#) · arXiv 2605.28816 · **Project page:** research.nvidia.com/labs/sil/projects/gamma-world

Most of the neural world models built so far are lonely places. They imagine a future one camera at a time: one player, one action stream, one viewpoint rolling forward into the next frame. That works for a single avatar wandering a generated landscape. It falls apart the moment you want what most real environments actually contain — several agents acting at once in the same space, each seeing the world from their own angle, each affecting what the others see. NVIDIA’s Gamma-World is a generative video world model built for exactly that case, and the interesting part for anyone outside graphics research is *how* it gets there: with two small architectural ideas that make “more agents” cheap instead of catastrophic.

The two problems with adding a second player

The obvious way to support multiple agents is to let every agent’s tokens attend to every other agent’s tokens — dense joint attention — and to tag each agent with a learned identity embedding so the model can tell player 1 from player 2. A recent multiplayer Minecraft model, Solaris, does roughly this and works fine for two players. But the approach carries two structural flaws that get worse as you scale.

First, dense cross-agent attention costs grow **quadratically** in the number of agents. Two players is tolerable; four or eight strangles a real-time rollout. Second, a learned per-slot identity embedding quietly assumes agents are *not* interchangeable. Two players with identical abilities get treated differently just because they landed in different slots, and the model is now welded to a fixed roster — you can’t add a fifth player without retraining. The paper’s framing insight is that agents in a shared world are intrinsically exchangeable, and a good architecture should bake that **permutation symmetry** in rather than fight it.

Marking agents without picking favorites

Gamma-World’s first move is **Simplex Rotary Agent Encoding**. Video transformers already use rotary position embeddings (RoPE) to inject where-and-when into attention by rotating query and key vectors along time, height, and width axes. Gamma-World adds a fourth axis — an agent axis — and the trick is what it puts there.

Instead of giving agent p a scalar phase (which strings agents along a one-dimensional line, so the gap between players 1 and 2 differs from the gap between 1 and 3), it places agents at the vertices of a regular simplex in rotary-angle space. Geometrically, every vertex sits at exactly equal distance from every other. Each agent gets a distinct, recognizable phase, but no agent is structurally special and every pair is treated identically. The encoding is parameter-free — no learned identity vectors — and it scales by selecting more vertices from a fixed pool. That last property is what enables the headline result: a model trained only on two-player data generates coherent four-player rollouts **zero-shot**, no retraining, no architecture change. During training they keep a pool of four vertices but only activate two per step, randomly sampling and permuting which two — so the model learns to read the simplex marker rather than memorize a slot order.

Routing interaction through a hub

The second move, **Sparse Hub Attention**, kills the quadratic cost. The observation behind it is pragmatic: in most shared-world settings, agents don't really need dense token-by-token gossip with each other at every layer. They mostly influence one another through a compact, evolving environment state. So instead of all-to-all attention, Gamma-World adds a small set of learnable “hub” tokens that act as a shared bulletin board. Each agent attends only to its own stream and to the hubs; the hubs attend to everyone and broadcast back. Cross-agent information flows along a two-hop path — agent to hub to agent — and the hubs are stripped from the output, so they're purely internal scratch space.

The payoff is that the dominant cross-agent attention cost drops from quadratic to **linear** in the number of agents. The efficiency curves bear this out: against dense attention, Sparse Hub Attention cuts both latency and FLOPs sharply as you go from two to four to eight players, where dense attention balloons. And it doesn't cost quality — in the ablations the full design matches or beats dense attention on fidelity and consistency while being far cheaper.

Making it run in real time

A world model nobody can interact with isn't much of a simulator, so the third piece is a training recipe aimed at low latency. A bidirectional diffusion *teacher* is trained for quality — it can see the whole sequence at once, which is great for consistency but useless for streaming. That teacher is then distilled into a block-causal *student* that generates one temporal block at a time with KV caching, so each agent reads its own cached past plus the cached hub state. The distilled few-step student streams action-responsive rollouts at **24 FPS**. Notably, cross-agent information still flows only through the hub even when running off cached histories, so the efficiency property survives into deployment.

Why an agent engineer should care

If you build or train multi-agent systems, the deliverable here isn't the pretty Minecraft video — it's a learned, controllable environment that several agents can act in simultaneously and that responds to their actions in real time. That's a training environment. Hand-built multi-agent simulators are expensive and brittle; a generative world model that stays consistent across agent perspectives, tracks a shared latent state even when agents leave each other's view, and scales to more agents without retraining is a plausible substitute for the simulation layer in reinforcement learning or embodied-agent pipelines. The paper underlines generality by treating a robot's two arms as two "agents" and modeling bimanual coordination with the same machinery that handles game players — the formulation doesn't care whether your agents are avatars, robot limbs, or software processes sharing a state.

The more portable lesson is the scaling trick itself. When you have N interacting entities and your instinct is all-to-all communication, ask whether a small shared hub can mediate instead — you often trade quadratic cost for linear with little loss, and the same hub-and-spoke idea transfers well beyond video. Pair that with an identity scheme that respects permutation symmetry, and you get the property everyone actually wants: train small, deploy bigger, no retraining. Worth keeping in mind that the authors flag honest limits — evaluation is mostly games plus a robotics demo, very large populations may outgrow a fixed rotary band, and without explicit physics, long rollouts can still drift.

If Gamma-World is about agents that share a world, the next chapter is about what happens to agents over *time* — how deployed agents quietly degrade across their own lifespan, and what "aging" looks like for systems you've already shipped.

Chapter 4: Your Agents Are Aging Too

Paper: [Your Agents Are Aging Too: Agent Lifespan Engineering for Deployed Systems](#) · arXiv 2605.26302 · **Project page:** agingbench.github.io

Your agent passed every test on launch day. Six weeks later it tells a user they’re still on the premium plan they canceled in January, drafts an email to the wrong John, and quietly forgets the recurring Tuesday therapy appointment after a routine memory cleanup. The model weights never changed. The benchmark score is irrelevant. Something else got old.

That something is what this paper names **agent aging**: time-dependent reliability decay in a deployed agent, driven not by the model but by everything wrapped around it. A long-lived agent is a harness — a frozen language model coupled to memory writing, a growing store, retrieval, prompts, and maintenance routines. Even with weights pinned, the agent’s effective state keeps shifting every time it compresses old history, accumulates similar memories, revises a fact, or undergoes a lifecycle event like recompaction. Reliability stops being a snapshot property of the base model and becomes a lifespan property of the whole system. Day-one evals can’t see it, because on day one nothing has aged yet.

Four ways an agent gets old

The authors organize decay into four mechanisms, each tied to a different deployment pressure.

Compression aging happens at write time: when an agent summarizes a session into a fixed budget, it has to decide what to keep before it knows what future queries will ask. Low-frequency specifics go first — dollar amounts, proper nouns, exact dosages — while the bland high-level summary survives. “Take 50 mg of metoprolol twice daily” decays into “you take a daily medication.”

Interference aging needs no information loss and no changed facts at all. As the store grows, similar entries crowd each other out at retrieval time. Two contacts named John Smith and John Smyth, two budgets in two domains — the right one gets buried under confusable neighbors.

Revision aging is the stale-fact problem: a value changes and the update doesn’t propagate. Its nastiest form is derived state, where an answer is accumulated from a running history (a budget that’s an initial value plus a stream of deltas). Miss one delta and every downstream answer is wrong, with compounding error that plain keyword recall never catches. **Maintenance aging** is different in kind — it’s triggered by an action taken *on* the agent. A recompaction, a prompt update, a history flush, a model rotation silently breaks something the agent reliably knew the day before.

To make this measurable, AgingBench generates multi-session task streams with a temporal dependency graph baked in: facts that supersede earlier facts, probes that depend on facts introduced many sessions apart, confusable entities that accumulate, and lifecycle events fired at controlled times. Across 7 scenarios, 14 models, multiple memory policies, and both runner-

controlled and autonomous agents, the team ran roughly 400 runs spanning 8 to 200 sessions, scoring not a single number but mechanism-specific aging curves over an operational lifetime.

Why one memory score lies to you

If you run agents in production, this is squarely your problem — and the paper’s central finding is a warning about your dashboards. Aging is multi-dimensional, and the dimensions move independently. The starkest example: an agent’s *behavioral* compliance can stay perfectly clean while its *factual* precision quietly collapses. In one lifestyle-assistant scenario, the explicit constraint-violation rate sat at zero across the whole horizon — the agent kept producing fluent, well-formed responses about budgets and preferences — while constraint precision fell from 0.90 to 0.37. The underlying values had been compressed away, but nothing tripped. Violation-based monitoring and uncertainty-based monitoring both miss this entirely. The failures look like confident, incorrect answers, not refusals or visible breaks.

Derived-state tracking is the other cliff. Bigger models didn’t reliably reduce accumulator error, and neither did swapping memory policies — the failure is representational, not a matter of capacity. If your agent maintains a running total over many updates and you’re hoping a stronger model fixes it, it won’t. You need explicit state maintenance or periodic recomputation.

And here’s the part that should change how you debug: the same aggregate failure rate hides completely different root causes. Across three models the total error rate clustered in a narrow band, yet the breakdown was wildly heterogeneous — one scenario was utilization-dominated, another write-dominated, and a third flipped between near-pure write failure and a large retrieval/interference component depending on the model. The generic prescription “give it more memory” is wrong in most of these cases.

What to monitor, and how to target the fix

AgingBench’s diagnostic trick is a counterfactual ablation ladder over the memory pipeline — write, retrieval, utilization. It runs three conditions: the agent’s own stack (baseline), then the agent’s memory store with an *oracle retriever* swapped in, then *oracle context* with gold facts injected directly. The accuracy gaps between these stages partition the error. Residual error even with gold facts in-context points to utilization — the model has what it needs and still fails to use it, a revision signature. Error that survives oracle retrieval but vanishes with oracle context points to write-time omission, a compression signature. Error that oracle retrieval alone recovers points to interference. Maintenance shock gets isolated temporally, by comparing the probe windows immediately before and after a lifecycle event.

The payoff is that the *same wrong answer* gets routed to a different repair. One scenario needed utilization-stage operators that force re-reads; another needed a value-preserving compaction prompt

at write time; a strong autonomous agent needed a planning-loop fix because it wrote lower-fidelity artifacts despite reasoning well over what it retrieved. None of those is “more memory.”

The practical takeaways for anyone shipping a persistent agent: stop trusting day-one scores as a reliability guarantee, because the harness ages even when the model doesn’t. Track reliability as a curve over sessions, not a single pass. Add mechanism-level probes — fact recall, not just behavioral compliance — because clean behavior dashboards routinely sit on top of decaying precision. And when something breaks, localize it to a pipeline stage before reaching for a fix, because writing, retrieval, and utilization failures each demand a different intervention.

That diagnostic instinct — refusing to accept “the agent forgot” and instead asking *which operation* caused it — is exactly where the next paper picks up. Chapter 5 turns to MemTrace, which traces an agent’s errors back to the specific memory operations that produced them.

Chapter 5: MemTrace — Observability for Agent Memory

Paper: [MemTrace: Tracing and Attributing Errors in Large Language Model Memory Systems](#) · arXiv 2605.28732 · **Code:** github.com/zjunlp/MemTrace

When an agent with a memory layer gives you a wrong answer, where did the failure actually happen? Not “the model hallucinated” — but which operation, in which session, broke the chain. A user states a preference. It gets stored correctly. Three sessions later an update silently overwrites it. Two sessions after that, the agent confidently answers using the stale value. The symptom surfaces at response time; the cause was buried turns ago, in a write operation nobody was watching. This is the problem the team behind MemTrace set out to make tractable, and their core move is one every engineer running a memory system should steal: stop treating memory as a black box and start treating it as a **traceable pipeline**.

Why memory failures are different

Debugging a stateless agent is comparatively easy. The failure lives inside the current run — a bad tool call, a wrong retrieval, a flawed reasoning step — and the trace is a short, readable log you can scan top to bottom. Memory-augmented agents break that comfort. They carry persistent state across interactions, so a fault introduced during an early construction, update, or deletion step can lie dormant and only detonate much later during retrieval or generation. The execution trace isn’t short, either: run a memory system across a long history and it can balloon to tens of megabytes.

Worse, a chronological log is the wrong shape for the job. It records operations in order, as a flat sequence pulled from different parts of the pipeline, but it can’t show you how a given memory variable was created, modified, overwritten, propagated, and finally consumed in a wrong prediction. The authors name this the **traceability gap**: failures are observable, but the faulty operations, when they were introduced, and how they propagated stay hidden. Existing memory benchmarks don’t help — they’re outcome-oriented, telling you whether a system stored or retrieved correctly, not recovering the causal path by which an error spread.

Turning a pipeline into a graph

MemTrace’s foundation is a representation change. Instrument the memory system’s source code, run it on a trajectory and a question, and every time it performs a memory update, a memory read, or an answer generation, record the variables involved, the operation applied, and the dependencies between them. The result is an **execution graph**: a directed acyclic bipartite graph whose nodes are variables (raw messages, retrieved memory units, intermediate summaries, prompts) and operations

(LLM inference, retrieval, filtering, parsing), with edges capturing information flow. Unlike a flat log, the graph explicitly captures which operations consume, modify, overwrite, or propagate each memory variable — so attribution can follow dependencies across turns and sessions instead of guessing from timestamps.

To build this for systems that weren't designed to be traced, the team wrote `smartcomment`, a lightweight tracing package that records developer-specified operations, variables, and their dependencies. They instrumented four representative systems — Long-Context, RAG, Mem0, and EverMemOS — ran them over sampled trajectories, collected 1,514 distinct errors, and had annotators label faulty operations, error types, and explanations. That yielded **MemTraceBench**: 160 human-verified failure cases, each with the QA pair, full execution trace, ground-truth error labels, faulty operation, and a human explanation.

Attribution as graph exploration

Given a failed graph and the golden answer, the goal is the *decisive error set* — the earliest, minimal cut of faulty operations whose correction would have rescued the run. MemTrace finds it by treating attribution as agentic graph exploration. It seeds a priority queue of variables (earliest timestamps first, so it inspects upstream causes before downstream symptoms), then at each step inspects only the local subgraph around the current variable — the operation's name, category, inputs, outputs, and dependencies — rather than loading the whole graph into context. If an operation looks locally correct, the agent follows the information flow downstream, adding the next variables to explore. It stops when it pins the decisive faulty operation. To start in the right neighborhood, it uses hybrid dense-plus-sparse retrieval (fused by reciprocal rank fusion) over the raw messages to pick a handful of likely-relevant starting points instead of dumping every input into the search space. Working context is kept in check with preview modes, pagination, and regex search so a single inspection can't blow up the token budget.

There's a cheaper variant, MemTrace-OBS, which flattens subgraphs into searchable operation blocks and lets the agent jump around via global search. It's far faster and lower-cost, but the structured graph walk wins on accuracy — and the gap is largest for smaller models. With GPT-4.1 mini, the disciplined graph exploration lifted overall error-type accuracy from 20.0% to 36.5%, because the constrained, follow-the-flow inspection stops a weak model from grabbing keywords off the golden answer and leaping straight to a wrong conclusion. The honest caveat: pinpointing the *exact* faulty operation is harder than naming the error type, with the best operation-identification accuracy landing around 46%.

What the traces reveal — and what to do with them

Run this across systems and the failures turn out to be systematic, clustered at the operation level: information loss during extraction, retrieval misalignment, response-stage aggregation slips. Each system has its own bottleneck. RAG has no extraction errors because it has no extraction module;

Mem0's extraction tends to keep high-level user info while dropping fine-grained details, and sometimes reassigns timestamps so content stays but its time shifts; EverMemOS fails more at reranking and the sufficiency checker. Notably, *every* system shows response errors — even when the right memories are retrieved, actually using them to answer well remains hard.

The payoff is the closed loop. Because attribution localizes the earliest decisive faulty operation, prompt optimization shrinks from an intractable whole-trajectory problem to a local one: run an off-the-shelf optimizer on just the few prompts feeding that operation, no replaying the full pipeline. Applied to Mem0, three rounds lifted end-task performance by **7.62%** — achieved despite attribution being imperfect, which is the encouraging part. Even noisy, graph-based credit assignment carries enough signal to drive real tuning.

The takeaway

If you operate an agent with a memory layer — RAG, Mem0, or anything that writes state across sessions — the durable lesson here is about altitude. The model is rarely the whole story; the failure usually lives in an operation somewhere in the construction-update-retrieval-response pipeline, and you can't fix what you can't trace. So instrument the memory layer the way you'd instrument any production system: record operations, variables, and their dependencies as a graph, not a flat log. Attribute concrete failures to the earliest decisive operation rather than blaming “the model.” Then close the loop — feed that attribution back into a local prompt fix for exactly the operation at fault. Observability for memory isn't a luxury feature; it's the difference between an agent you can debug and one you can only apologize for.
